



CC5061NI - Applied Data Science

60% Individual Coursework

2023-24 Autumn

Student Name: Sardul Ojha

London Met ID: 22067311

College ID: NP01AI4A220030

Assignment Due Date: Monday, May 13, 2024

Assignment Submission Date: Monday, May 13, 2024

Word Count: 2994

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Acknowledgement

I would like to express my sincere gratitude to everyone who has helped me in the completion of this project. Firstly, I extend my heartfelt thanks to my friends whose support and encouragement have been invaluable throughout this project. Their insights and feedback have greatly enriched the quality of my project.

I am also deeply grateful to Mr. Dipeshor Sir for his guidance and mentorship. His expertise in the field of Data Analytics has been instrumental in shaping the content and structure of this project. His constructive criticism and advice have significantly enhanced the clarity and depth of data analysis.

Furthermore, I would like to extend my appreciation to the college for providing us with the opportunity to work on such a challenging and rewarding real-time project. The experience gained from this project has been invaluable in developing our skills and preparing us for future endeavors in the field of data.

In conclusion, I express my sincere thanks to all those who have contributed to the successful completion of this project. Your support and guidance have been instrumental in achieving my goals, and for that, I am truly grateful.

Table of Contents

1. Introduction *1*
1.1. Aim **1**
1.2. Objectives **1**
1.3. Tools and Libraries Used **2**
2. Data Understanding *6*
3. Data Preparation *8*
4. Data Analysis *13*
5. Data Exploration *15*
6. Conclusion *23*
7. References *24*

Table of Figures

Figure 1: Logo of Python 2
Figure 2: Logo of Anaconda 2
Figure 3: Logo of jupyter 3
Figure 4: Logo of pandas 4
Figure 5: Logo of Matplotlib 4
Figure 6: Logo of MS Word 5
Figure 7: Screenshot of importing libraries 8
Figure 8: Screenshot of loading data and displaying first fire rows 8
Figure 9: : Screenshot of dropping columns and displaying 9
Figure 10: Screenshot of dropping na(missing) values 9
Figure 11: Screenshot of handling duplicates 10
Figure 12: Screenshot of displaying unique values from each column 11
Figure 13: Screenshot of renaming experience level column and displaying 12
Figure 14: Screenshot of displaying summary statistics 13
Figure 15: Screenshot of correlation 14
Figure 16: Screenshot of top 15 jobs 15
Figure 17: Screenshot of highest salary jobs 17
Figure 18: Screenshot of average salary based on experience 19
Figure 19: Screenshot of histogram of salary in usd 21
Figure 20: Screenshot of boxplot of salary in usd 22

Table of Tables

Table 1: Description of Columns 7

1. Introduction

In today's data-driven world, the demand for skilled professionals who can extract insights, generate models, or use data in some form is ever-growing. They look at tons of information and figure out how to use it to make businesses better. By studying data, they can predict what might happen in the future, make things work more efficiently, and even solve big problems like climate change and healthcare issues. So, these data jobs aren't just about making companies better, they are also about making the world a better place for everyone.

Understanding what affects how much money people in these data jobs make is super important. It's not just useful for people who want to be data experts, but also for companies that want to hire and keep the best talent. By looking closely at what influences the salaries, we can learn a lot about how pay works. This knowledge is also really important for how companies hire, keep their employees happy, and make sure everyone gets paid fairly in the world of data-driven jobs.

1.1. Aim

The main aim of this coursework is to conduct a comprehensive analysis of factors influencing salaries of data and AI related jobs using python programming language and its libraries.

1.2. Objectives

- To explore and understand the characteristics of the dataset.
- To preprocess and clean the dataset by removing unnecessary columns, handling missing values, checking for duplicates and so on.
- To perform statistical analysis on the dataset.
- To visually explore the dataset through different graphs and charts.

1.3. Tools and Libraries Used

a) Python

Python is an object-oriented, interpreted, and high-level programming language. It's easy and simple syntax makes it more readable and beginner friendly. Python comes with a large standard library that supports various tasks. It is most commonly used for developing websites, task automation, data analysis and visualization, machine learning, and artificial intelligence (Python Software Foundation, 2001-2023)



Figure 1: Logo of Python

Use:

- It is used as a programming language for the entire data analysis process.

b) Anaconda

Anaconda is a python distribution which is specifically for machine learning and data science. It comes with number of pre-installed packages like SciPy, Malplotlib, Pandas, and NumPy. It is also designed to work seamlessly with Jupyter Notebooks (Simplilearn Solutions, 2009-2024).



Figure 2: Logo of Anaconda

Use:

- It provides essential data science libraries and tools such as pandas, matplotlib without any manual installation.

c) Jupyter Notebook

The Jupyter Notebook is a web application for creating and sharing computational documents. You can write code in more than 40 programming languages, including Python.We can use big data tools like Apache Spark with Python, R, and Scala and also analyze data with tools like pandas, sci-kit learn and so on (Jupyter, 2024).



Figure 3: Logo of jupyter

Use:

- It is used as an interactive environment where Python code can be executed in a cell-by-cell manner for analysis process and its output.

d) Pandas

Pandas is a python library which mainly is used for managing datasets. It offers various functions for data analysis, cleaning, exploration, and manipulation. This library helps in the analysis of large datasets and the derivation of conclusions through statistical methodologies. With Pandas, messy datasets can be cleaned, making them meaningful and relevant (Refsnes Data, 1999-2024).



Figure 4: Logo of pandas

Use:

- It is used for loading the dataset into a DataFrame, handling missing values, removing unnecessary columns, and performing various data related querries.

e) Matplotlib

Matplotlib is a powerful libaray in python for making all sorts of graphs and charts. We can create high-quality plots for reports or presentations. It's easy to customize how our graphs look, and you can save them in different file formats. It also works well with other tools like JupyterLab, Jupyter Notebook and different graphical interfaces (Matplotlib, 2012-2023).



Figure 5: Logo of Matplotlib

Use:

- It is used for creating visualizations such as bar graphs, histograms, and box plots.

f) MS Word

Microsoft released MS Word, a program for making documents, on October 25, 1983. It helps create professional stuff like letters and reports. With its many features, we can easily edit and format files and documents (BYJU'S, 2021).



Figure 6: Logo of MS Word

Use:

- It is used for writing the report of the program.

2. Data Understanding

The dataset contains information about employees working in data-related jobs, including their experience level, job titles, salaries, and employment details like company size and remote work options. By understanding this data, we can see how salaries vary based on factors like experience level and job title. We can also explore how things like where the employee lives or the size of the company they work for affect their salary and whether they can work remotely. This helps businesses make decisions about hiring, compensation, and creating work environments that attract and retain talent in the data field.

The column description of each column present in the dataset is given in the table below:

S. N	Column Name	Description	Data Type
1	work_year	This column provides the year of the data-related job. It ranges from 2020 to 2023.	Integer
2	experience_level	This column contains values indicating the experience level i.e. “SE” for senior level, “MI” for medium level, “EN” for Entry Level and “EX” for executive level.	Object
3	employment_type	The 'employment_type' column contains values indicating the type of employment namely 'FT' for full-time, 'CT' for contract, 'FL' for freelance, and 'PT' for part-time.	Object
4	job_title	This column specifies the titles or designations associated with each employee.	Object
5	salary	The ‘salary’ column’ provides the salary(number) of the job. The salary might be in any currency.	Integer
6	salary_currency	This column contains the currency of salary in ‘salary’ column. For example, if the	Object

		currency in ‘salary’ column is in US Dollars then the value in this column is “USD”.	
7	salary_in_usd	This column contains all the values of ‘salary’ column converted to USD. The USD salary in ‘salary’ column remains the same whereas other currencies are converted respectively.	Integer
8	employee_residence	The 'employee_residence' column holds the country of residence of the employee, with values represented in the short form of their respective country names.	Object
9	remote_ratio	This column indicates the extent of remote work allowed for each job listing. It contains values of 0, 50, or 100, representing no remote work, partial remote work (50%), or full remote work (100%) respectively.	Integer
10	company_location	This column holds the country of residence of the company, with values represented in the short form of their respective country names similar to ‘employee_residence’ column.	Object
11	company_size	The "company_size" column categorizes companies based on their size. It includes values such as "L" for large, "M" for medium, and "S" for small.	Object

Table 1: Description of Columns

3. Data Preparation

Before proceeding to data preparation, data analysis, and data exploration, all the necessary libraries required for these processes are imported. We imported the pandas library for data preparation, data cleaning, and analysis. The "pd" alias is used for this library. Similarly, the pyplot module of the matplotlib library is imported for data visualization and exploration. The 'plt' alias is used for this module. Finally, the warnings module is imported, and the filterwarnings function is called with 'ignore' as its parameter to ignore(remove) all the warnings that occur in the process.

```
In [1]: # importing libraries

        import pandas as pd
        import matplotlib.pyplot as plt
        import warnings

        warnings.filterwarnings('ignore')
```

Figure 7: Screenshot of importing libraries

a. Write a python program to load data into pandas DataFrame.

```
In [2]: # Reading and displaying the dataset

        df = pd.read_csv("data_science_salaries.csv")
        df.head()
```

Out[2]:

	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	com
0	2023	SE	FT	Principal Data Scientist	80000	EUR	85847	ES	100	ES	
1	2023	MI	CT	ML Engineer	30000	USD	30000	US	100	US	
2	2023	MI	CT	ML Engineer	25500	USD	25500	US	100	US	
3	2023	SE	FT	Data Scientist	175000	USD	175000	CA	100	CA	
4	2023	SE	FT	Data Scientist	120000	USD	120000	CA	100	CA	

Figure 8: Screenshot of loading data and displaying first fire rows

Pandas' read_csv method is called to read the CSV file. The name of the CSV file, i.e., "data_science_salaries.csv", is passed as a parameter, and it is stored in the variable df as a DataFrame (a 2D labeled data structure). Finally, the head method is called to display the first five rows of the dataframe. As the rows are displayed, we can confirm that data is loaded into the pandas dataframe.

b. Write a python program to remove unnecessary columns i.e., salary and salary currency.

```
In [7]: # Removing unnecessary columns

        df = df.drop(['salary', 'salary_currency'], axis = 1)
        df.columns

Out[7]: Index(['work_year', 'experience_level', 'employment_type', 'job_title',
               'salary_in_usd', 'employee_residence', 'remote_ratio',
               'company_location', 'company_size'],
              dtype='object')
```

Figure 9: : Screenshot of dropping columns and displaying

The drop method is used to remove rows or columns from a DataFrame. It takes a list of rows or columns and an axis as parameters. Axis 0 represents rows, whereas axis 1 represents columns. Here, we have passed the 'salary' and 'salary_currency' columns in a list, and the value of the axis is set to 1. By displaying the columns using the columns attribute, we can see that the specified columns have been removed from the dataframe.

c. Write a python program to remove the NaN missing values from updated dataframe.

```
In [8]: # Removing missing values (NaN)

        df = df.dropna()
```

Figure 10: Screenshot of dropping na(missing) values

The dropna method of pandas dataframe is used to drop(remove) the missing values from the dataframe. Here, the dropna method is called and old dataframe (df) is set to new dataframe without NaN missing values.

d. Write a python program to check duplicates value in the dataframe.

```
In [9]: # Checking for duplicates
        df.duplicated().sum()
Out[9]: 1171

In [10]: # Dropping duplicate values
         df = df.drop_duplicates()

In [11]: # Checking again for duplicates
         df.duplicated().sum()
Out[11]: 0
```

Figure 11: Screenshot of handling duplicates

The duplicated method returns a boolean series of the DataFrame where the values are 'True' for duplicated and 'False' for unique. The sum method counts all the 'True' values from the DataFrame and gives the number of duplicated rows, i.e., 1171. Then, the drop_duplicates method is called to remove (drop) all the duplicate values from the DataFrame. Finally, upon rechecking, there are no duplicate values present.

e. Write a python program to see the unique values from all the columns in the dataframe.

```
In [12]: # Displaying unique values for each column

         # Empty dict to store unique values
         unique_values = {}

         # Adding column as keys and their unique values in the dict
         for column in df.columns:
             unique_values[column] = df[column].unique()

         # Displaying
         for column, values in unique_values.items():
             print(f"{column}: {values}")
             print("-" * 90)

work_year: [2023 2022 2020 2021]
------------------------------------------------------------------------------------------
experience_level: ['SE' 'MI' 'EN' 'EX']
------------------------------------------------------------------------------------------
employment_type: ['FT' 'CT' 'FL' 'PT']
------------------------------------------------------------------------------------------
job_title: ['Principal Data Scientist' 'ML Engineer' 'Data Scientist'
 'Applied Scientist' 'Data Analyst' 'Data Modeler' 'Research Engineer'
 'Analytics Engineer' 'Business Intelligence Engineer'
 'Machine Learning Engineer' 'Data Strategist' 'Data Engineer'
 'Computer Vision Engineer' 'Data Quality Analyst'
 'Compliance Data Analyst' 'Data Architect'
 'Applied Machine Learning Engineer' 'AI Developer' 'Research Scientist'
 'Data Analytics Manager' 'Business Data Analyst' 'Applied Data Scientist'
 'Staff Data Analyst' 'ETL Engineer' 'Data DevOps Engineer' 'Head of Data'
 'Data Science Manager' 'Data Manager' 'Machine Learning Researcher'
 'Big Data Engineer' 'Data Specialist' 'Lead Data Analyst'
 'BI Data Engineer' 'Director of Data Science'
 'Machine Learning Scientist' 'MLOps Engineer' 'AI Scientist'
 'Autonomous Vehicle Technician' 'Applied Machine Learning Scientist'
 'Lead Data Scientist' 'Cloud Database Engineer' 'Financial Data Analyst'
 'Data Infrastructure Engineer' 'Software Data Engineer' 'AI Programmer'
 'Data Operations Engineer' 'BI Developer' 'Data Science Lead'
 'Deep Learning Researcher' 'BI Analyst' 'Data Science Consultant'
 'Data Analytics Specialist' 'Machine Learning Infrastructure Engineer'
 'BI Data Analyst' 'Head of Data Science' 'Insight Analyst'
 'Deep Learning Engineer' 'Machine Learning Software Engineer'
 'Big Data Architect' 'Product Data Analyst'
 'Computer Vision Software Engineer' 'Azure Data Engineer'
 'Marketing Data Engineer' 'Data Analytics Lead' 'Data Lead'
 'Data Science Engineer' 'Machine Learning Research Engineer'
 'NLP Engineer' 'Manager Data Management' 'Machine Learning Developer'
 '3D Computer Vision Researcher' 'Principal Machine Learning Engineer'
 'Data Analytics Engineer' 'Data Analytics Consultant'
 'Data Management Specialist' 'Data Science Tech Lead'
 'Data Scientist Lead' 'Cloud Data Engineer' 'Data Operations Analyst'
 'Marketing Data Analyst' 'Power BI Developer' 'Product Data Scientist'
 'Principal Data Architect' 'Machine Learning Manager'
 'Lead Machine Learning Engineer' 'ETL Developer' 'Cloud Data Architect'
 'Lead Data Engineer' 'Head of Machine Learning' 'Principal Data Analyst'
 'Principal Data Engineer' 'Staff Data Scientist' 'Finance Data Analyst']
------------------------------------------------------------------------------------------
salary_in_usd: [ 85847  30000  25500 ...  28369 412000  94665]
------------------------------------------------------------------------------------------
employee_residence: ['ES' 'US' 'CA' 'DE' 'GB' 'NG' 'IN' 'HK' 'PT' 'NL' 'CH' 'CF' 'FR' 'AU'
 'FI' 'UA' 'IE' 'IL' 'GH' 'AT' 'CO' 'SG' 'SE' 'SI' 'MX' 'UZ' 'BR' 'TH'
 'HR' 'PL' 'KW' 'VN' 'CY' 'AR' 'AM' 'BA' 'KE' 'GR' 'MK' 'LV' 'RO' 'PK'
 'IT' 'MA' 'LT' 'BE' 'AS' 'IR' 'HU' 'SK' 'CN' 'CZ' 'CR' 'TR' 'CL' 'PR'
 'DK' 'BO' 'PH' 'DO' 'EG' 'ID' 'AE' 'MY' 'JP' 'EE' 'HN' 'TN' 'RU' 'DZ'
 'IQ' 'BG' 'JE' 'RS' 'NZ' 'MD' 'LU' 'MT']
------------------------------------------------------------------------------------------
remote_ratio: [100   0  50]
------------------------------------------------------------------------------------------
company_location: ['ES' 'US' 'CA' 'DE' 'GB' 'NG' 'IN' 'HK' 'NL' 'CH' 'CF' 'FR' 'FI' 'UA'
 'IE' 'IL' 'GH' 'CO' 'SG' 'AU' 'SE' 'SI' 'MX' 'BR' 'PT' 'RU' 'TH' 'HR'
 'VN' 'EE' 'AM' 'BA' 'KE' 'GR' 'MK' 'LV' 'RO' 'PK' 'IT' 'MA' 'PL' 'AL'
 'AR' 'LT' 'AS' 'CR' 'IR' 'BS' 'HU' 'AT' 'SK' 'CZ' 'TR' 'PR' 'DK' 'BO'
 'PH' 'BE' 'ID' 'EG' 'AE' 'LU' 'MY' 'HN' 'JP' 'DZ' 'IQ' 'CN' 'NZ' 'CL'
 'MD' 'MT']
------------------------------------------------------------------------------------------
company_size: ['L' 'S' 'M']
------------------------------------------------------------------------------------------
```

Figure 12: Screenshot of displaying unique values from each column

At first, an empty dictionary named 'unique_values' is created to store the unique values. Now, a for loop is used to iterate over every column in the dataframe. The column name is used as a key, and the 'unique' method is used to obtain the unique values, which are then stored in the dictionary. Finally, all the

keys and values, i.e. columns and their corresponding unique values, are displayed.

f. Rename the experience level columns as below.

SE – Senior Level/Expert

MI – Medium Level/Intermediate

EN – Entry Level

EX – Executive Level

```
In [28]: # Renaming values in the experience level column

         df = df.replace({'experience_level': {'SE' : 'Senior Level/Expert',
                                               'MI' : 'Medium Level/Intermediate',
                                               'EN' : 'Entry Level',
                                               'EX' : 'Executive Level'}})

         # Displaying experience_level column
         df[['experience_level']]
```

Out[28]:

	experience_level
0	Senior Level/Expert
1	Medium Level/Intermediate
2	Medium Level/Intermediate
3	Senior Level/Expert
4	Senior Level/Expert
...	...
3750	Senior Level/Expert
3751	Medium Level/Intermediate
3752	Entry Level
3753	Entry Level
3754	Senior Level/Expert

2584 rows × 1 columns

Figure 13: Screenshot of renaming experience level column and displaying

The replace function of the pandas DataFrame takes a nested dictionary as a parameter. The key of the outer dictionary represents the column whose values are to be replaced. The key of the inner dictionary represents the current value, and its corresponding value is the value to be replaced with. Here, respective values are passed in the nested dictionary to replace the values. Finally, the 'experience_level' column is displayed to check if the values have been replaced or not.

4. Data Analysis

a. Write a Python program to show summary statistics of sum, mean, standard deviation, skewness, and kurtosis of any chosen variable.

```
In [14]: # Displaying sum, mean, standard deviation, skewness, and kurtosis

         # chosen column
         column = "salary_in_usd"

         # Displaying values
         print("SUMMARY STATS")
         print("-------------")
         print("Sum: " + str(round(df[column].sum(), 2)))
         print("Mean: " + str(round(df[column].mean(), 2)))
         print("Standard Deviation: " + str(round(df[column].std(), 2)))
         print("Skewness: " + str(round(df[column].skew(), 2)))
         print("Kurtosis: " + str(round(df[column].kurt(), 2)))

         SUMMARY STATS
         -------------
         Sum: 344729580
         Mean: 133409.28
         Standard Deviation: 67136.84
         Skewness: 0.62
         Kurtosis: 0.83
```

Figure 14: Screenshot of displaying summary statistics

At first, the name of the column is stored in the variable 'column'. Here, the column is 'salary_in_usd. Then, the sum, mean, standard deviation, skewness, and kurtosis are calculated using their respective functions. The values are rounded to two decimal places and converted into strings to print the result.

b. Write a Python program to calculate and show correlation of all variables.

```
In [15]: # Correlation of numeric variables

         df.corr()
```

Out[15]:

	work_year	salary_in_usd	remote_ratio
work_year	1.000000	0.236958	-0.219160
salary_in_usd	0.236958	1.000000	-0.084502
remote_ratio	-0.219160	-0.084502	1.000000

```
In [16]: # Visualizing correlation in heatmap

         plt.imshow(df.corr(), cmap='coolwarm')
         plt.colorbar(label='Value')
         plt.title('Correlation Heatmap', fontsize=17)
         plt.xticks([])
         plt.yticks([])

         plt.show()
```

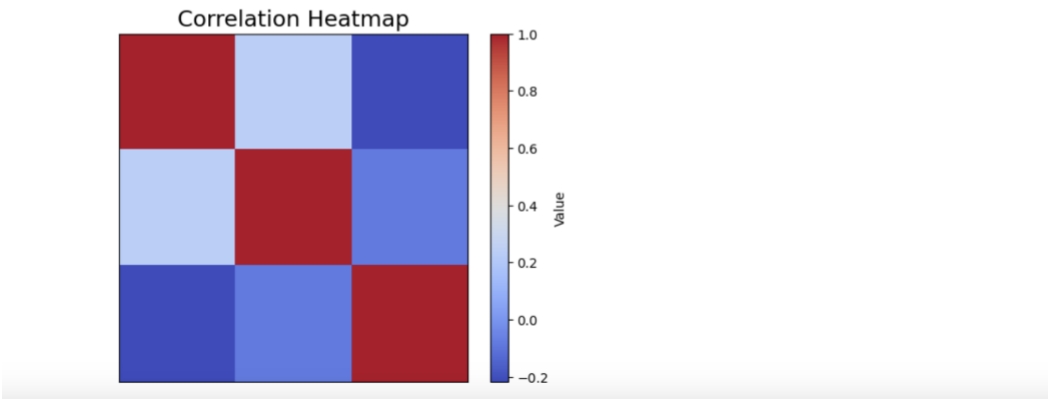


Figure 15: Screenshot of correlation

To display the correlation of all the variables, the simple pandas method 'corr()' is used. Here, the correlation of all the numeric columns is shown in the correlation table.

Using imshow method, the correlation table is visualized in a heatmap. From the correlation table or heatmap, we can see that there is low positive correlation between work_year and salary_in_usd column. Similarly, work_year and remote_ratio columns have low negative correlation whereas salary_in_usd and remote_ratio columns have very low negative correlation.

5. Data Exploration

a. Write a python program to find out top 15 jobs. Make a bar graph of jobs as well.

```
In [16]: # Top 15 jobs according to their count

         top_15_jobs = df['job_title'].value_counts().head(15)
         top_15_jobs
```

```
Out[16]: Data Engineer                598
         Data Scientist               538
         Data Analyst                 396
         Machine Learning Engineer    206
         Analytics Engineer            91
         Research Scientist            65
         Data Architect                64
         Data Science Manager          52
         ML Engineer                   34
         Research Engineer             33
         Applied Scientist             31
         Machine Learning Scientist    26
         Data Science Consultant       23
         Data Manager                  23
         Computer Vision Engineer      18
         Name: job_title, dtype: int64
```

```
In [17]: # Showing top 15 jobs in bar graph

         plt.bar(top_15_jobs.index, top_15_jobs.values,
                 color = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
                    '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf',
                    '#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd'])

         plt.title("Top 15 Jobs", fontsize=17)
         plt.xlabel ("Position")
         plt.ylabel("Number of Jobs")
         plt.xticks(rotation=90)
         plt.grid(axis='y', linestyle='--', alpha=0.7)

         plt.show()
```

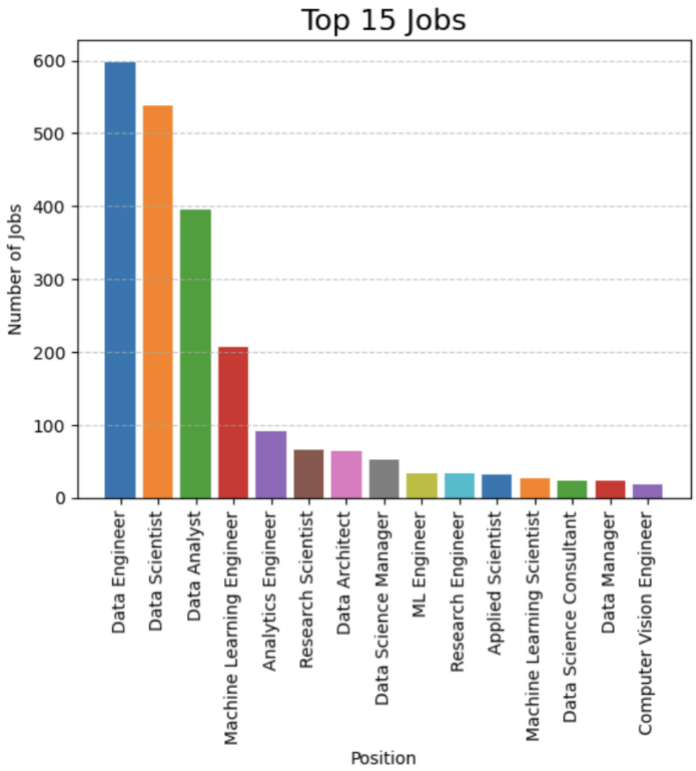


Figure 16: Screenshot of top 15 jobs

At first, the number of occurrences of each job title is counted using the 'value_counts' method, and the top 15 jobs are stored as a series in the 'top_15_jobs' variable.

To create a bar graph, we have used the 'bar' method from matplotlib's pyplot library. The index and values of the above series, i.e., jobs and their corresponding counts, are passed as parameters. Similarly, a list of colors is also passed to provide attractive colors to the bar graph. Titles, x-labels, y-labels, and other attributes are set using their respective methods. Finally, the 'show' method is called to display the bar graph.

From the bar graph, we can visualize that data engineer there is highest number (about 600) of data engineer jobs in the given dataset. It is followed by data scientist, data analyst and machine learning engineer. These top 4 jobs have count greater than 200. Rest of the jobs like analytics engineer, research scientist, data architect and so on are present in less than 100.

b. Which job has the highest salaries? Illustrate with bar graph.

```
In [31]: # Job with the highest salary

         df.loc[df['salary_in_usd'] == df['salary_in_usd'].max()]
```

Out[31]:

	work_year	experience_level	employment_type	job_title	salary_in_usd	employee_residence	remote_ratio	company_location	company_size
3522	2020	Medium Level/Intermediate	FT	Research Scientist	450000	US	0	US	M

```
In [32]: # Top 10 jobs with highest maximum salaries

         top_10_highest_salary_jobs = df.groupby('job_title')['salary_in_usd'].max().sort_values(ascending=False).head(10)
         top_10_highest_salary_jobs
```

Out[32]:

```
job_title
Research Scientist                    450000
Data Analyst                          430967
AI Scientist                          423834
Applied Machine Learning Scientist    423000
Principal Data Scientist              416000
Data Scientist                        412000
Data Analytics Lead                   405000
Applied Data Scientist                380000
Data Architect                        376080
Data Science Tech Lead                375000
Name: salary_in_usd, dtype: int64
```

```
In [20]: # Showing top 10 jobs with highest salaries in bar graph

         plt.bar(top_10_highest_salary_jobs.index, top_10_highest_salary_jobs.values,
                 color = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
                    '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf'])

         plt.title("Top 10 Jobs", fontsize=17)
         plt.xlabel("Position")
         plt.ylabel ("Average Salary ($)")
         plt.xticks(rotation=90)

         plt.show()
```

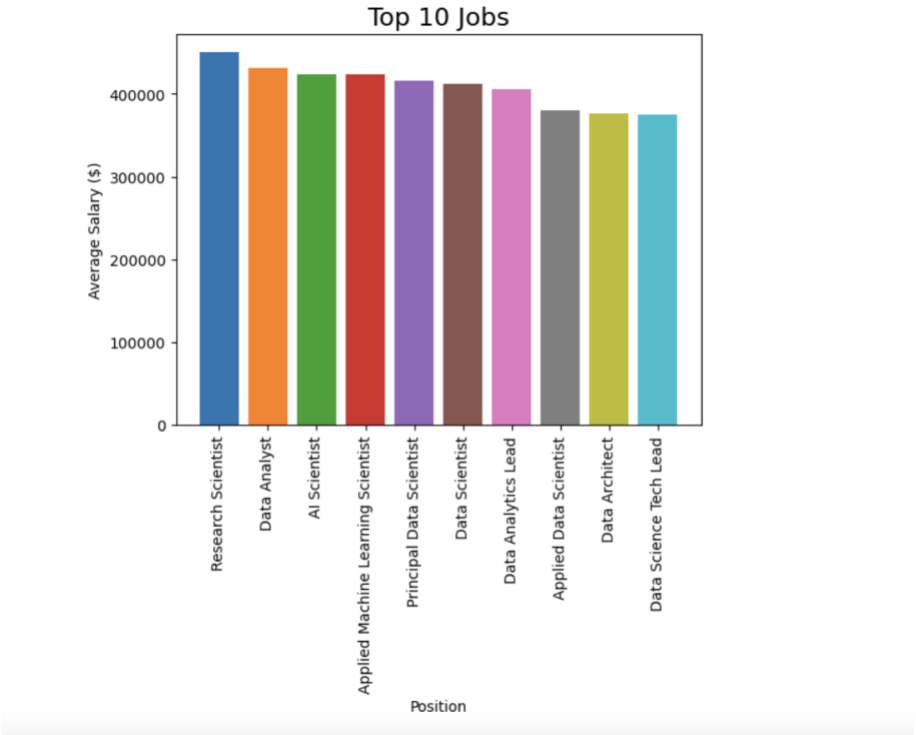


Figure 17: Screenshot of highest salary jobs

Using the groupby function, job titles are grouped according to their maximum salary in USD. The 'sort_values' method is used, and the ascending parameter is set to false to arrange them in descending order. Finally, the 'head(10)' method is used to retrieve the top 10 jobs with the highest salaries. This pandas series is stored in the 'top_10_highest_salary_jobs' variable.

Now, the index and values, i.e. jobs and their maximum salaries, are passed as parameters to the 'bar' method of matplotlib's pyplot library to plot a bar graph. Respective methods are used for title, xlabel, ylabel, and finally, the bar graph is displayed using the 'show' method.

From the above results, we can conclude that research scientist is the job with highest salary i.e. 450000 USD$. It is followed by data analyst, AI scientist and others. However, there is not much difference in the highest salaries of these jobs.

c. Write a python program to find out salaries based on experience level. Illustrate it through bar graph.

```
In [21]: # Average salary based on experience

         exp_avg_salary = df.groupby('experience_level')['salary_in_usd'].mean()
         exp_avg_salary
```

```
Out[21]: experience_level
         Entry Level                  72648.685185
         Execute level               191078.208333
         Medium Level/Intermediate   101828.783133
         Senior Level/Expert         153897.435650
         Name: salary_in_usd, dtype: float64
```

```
In [22]: # Showing average salary based on experience salaries in bar graph

         plt.bar(exp_avg_salary.index, exp_avg_salary.values,
                 color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728'])

         plt.title("Salaries Based On Experience", fontsize=17)
         plt.xlabel("Experience Level")
         plt.ylabel("Average Salary ($)")
         plt.xticks(rotation=15)
         plt.grid(axis='y', linestyle='--', alpha=0.7)

         plt.show()
```

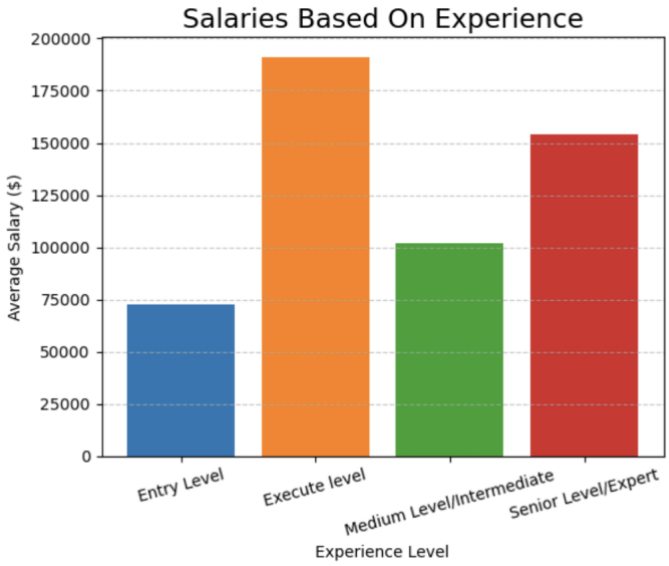


Figure 18: Screenshot of average salary based on experience

Here, the groupby function is used to group the 'experience_level' using mean as the aggregate function. This series is stored int exp_avg_salary variable. Again, the bar method is called passing index and of the above series as parameter to plot the bar graph. The title, labels, colors, and other attributes are given by using their respective functions. Finally, "show" function displays the bar graph.

From the above graph, we can say that higher the position, higher the salary. Execute level have the highest average salary due to their extensive experiences, leadership responsibilities, and strategic roles in the organization. It is followed by senior/ expert level as they are particularly specialized in a field, more experience, and advanced qualifications. Then comes medium level/ intermediate with average salary of about 100,000 USD$ and then finally entry level experience with about 72,000 USD$ average salary.

d. Write a Python program to show histogram and box plot of any chosen different variables. Use proper labels in the graph.

```
In [24]: # Histogram

         plt.figure(figsize=(10, 6))

         bins = int((df['salary_in_usd'].max() - df['salary_in_usd'].min()) / 20000)
         plt.hist(df['salary_in_usd'], bins=bins, color='skyblue', edgecolor='black')
         plt.title("Histogram", fontsize=17)
         plt.xlabel("Salary in USD")
         plt.ylabel("Frequency")

         plt.show()
```

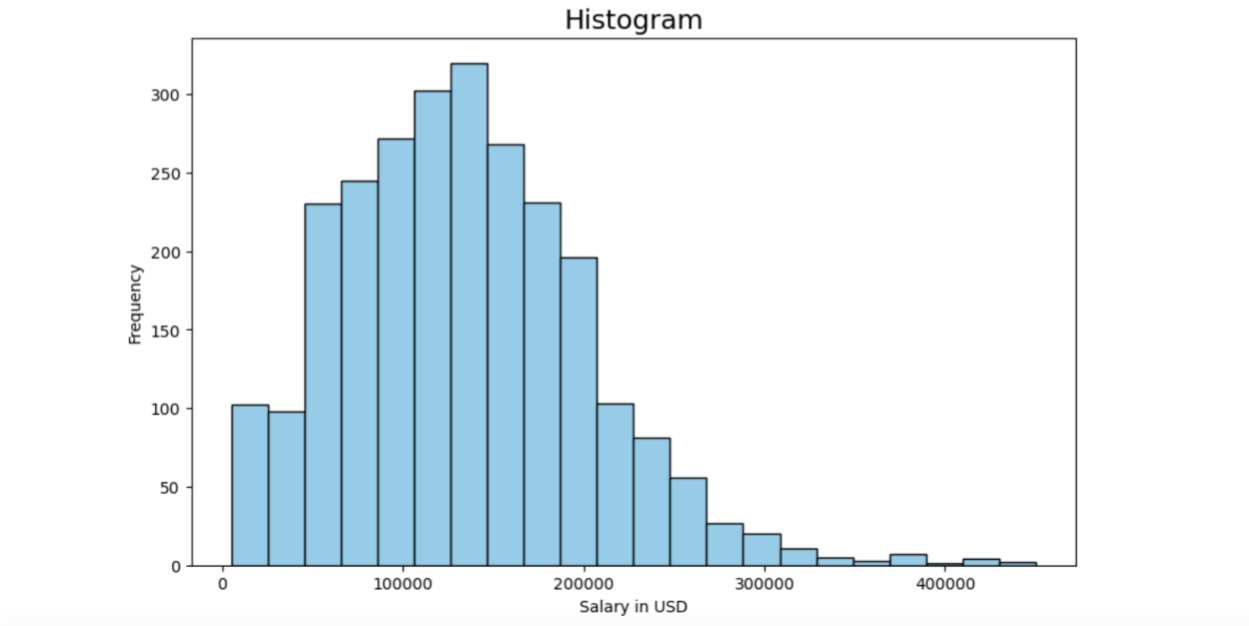


Figure 19: Screenshot of histogram of salary in usd

The plt.figure() function called at the top with figure size 10 by 6 given at parameters determines the dimensions of the figure of the graph. Range of salary_in_usd column is divided by 20000(bin width) , its integer value is set to bins variable. It determines the number of bins. Now, hist function of plt library is called with respective parameters to plot the historgram of salary_in_usd column. The title, labels, colors, and other attributes are given by using their respective functions. Finally, “show” function displays the histogram.

According to this histogram, the frequency of salary between 100000 to 200000 is the highest. It also shows that the distribution is right skewed which means that there are relatively more jobs with lower salaries and a few jobs with very high salaries.

```
In [25]: # Box Plot

plt.boxplot(df['salary_in_usd'], patch_artist=True, boxprops=dict(facecolor='brown'))
plt.title("Box Plot", fontsize=17)
plt.xlabel ("Salary in USD")

plt.show()
```

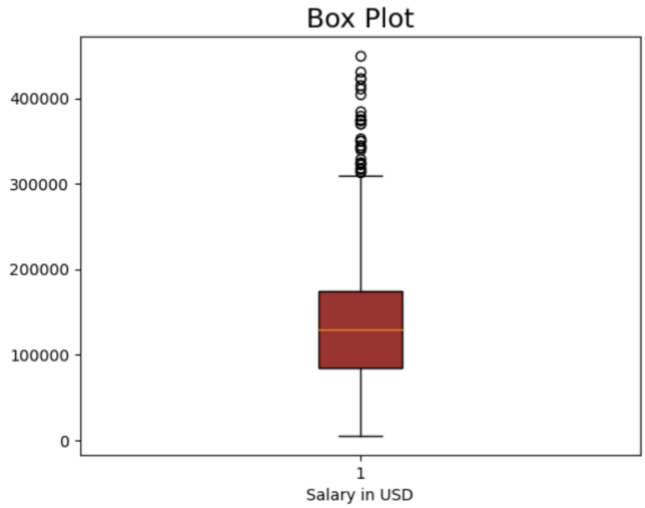


Figure 20: Screenshot of boxplot of salary in usd

Similarly, the boxplot function of matplotlib library is used for plotting the boxplot. Passing the column (salary_in_usd in our case), patch_artist and boxprops for color, creates our boxplot. The title and xlabel are given respectively and finally plt.show displays the boxplot for salary_in_usd.

In the displayed boxplot, we can see that the median salary in usd lies between 100000 and 200000 USD$. Similarly, the upper quartile also lies below 200000 USD$ and the lower quartile is below 100000 USD$. The outliers shown in the boxplot are the jobs with exceptionally high salary.

6. Conclusion

A thorough evaluation of the python notebook has been conducted in this report, and all the findings have been found to be satisfactory. All the data wrangling that was carried out were successful, and all the project's objectives outlined in the coursework were met, with all the errors and bugs found being fixed.

This coursework has been a valuable learning experience, helping me gain knowledge, skills, and experience. I became much more familiar with pandas and matplotlib, and other tools such as Anaconda-Navigator, Jupyter Notebook, MS Word, and many more. This real-world scenario problem also deepened my understanding of data analytics is done. Additionally, this project enhanced my research and report writing skills. All these skills and knowledge will be useful in my future endeavors.

Many challenges were encountered and solved during my coursework journey. I had to seek help from my teachers and friends to solve the problems faced. Similarly, I also faced difficulties during the report, such as formatting it correctly, crashing of MS Word and Eclipse IDE. I had to rely on Google for solutions and learned the importance of making backups after completing a major portion of the program and report.

Finally, this project has been a challenging but rewarding experience. I am proud that I was able to complete this project, and I would like to thank everyone, especially all my teachers, who helped me make this project a success. It was a great learning experience, and I am grateful to have been a part of it.

7. References

Python Software Foundation, 2001-2023. *What is python?.* [Online]
Available at: https://www.python.org/doc/essays/blurb/
[Accessed 11 May 2023].
BYJU'S, 2021. *Byjus.* [Online]
Available at: https://byjus.com/govt-exams/microsoft-word/
Simplilearn Solutions, 2009-2024. *https://www.simplilearn.com/anaconda-vs-python-article#:~:text=While%20Python%20is%20a%20versatile,and%20resolve%20conflicts%20between%20packages..* [Online]
Available at: https://www.simplilearn.com/anaconda-vs-python-article#:~:text=While%20Python%20is%20a%20versatile,and%20resolve%20conflicts%20between%20packages.
[Accessed 23 05 2023].
Jupyter, 2024. *Jupyter Notebook.* [Online]
Available at: https://jupyter.org
Refsnes Data, 1999-2024. *Pandas Introduction.* [Online]
Available at: https://www.w3schools.com/python/pandas/pandas_intro.asp
Matplotlib, 2012-2023. *Matplotlib: Visualization with Python.* [Online]
Available at: https://matplotlib.org
[Accessed 05 05 2023].